IntellMeet

AI-Powered Enterprise Meeting & Collaboration Platform

Real-time video • AI summaries • Smart action items • Team collaboration

Item	Details
Project	Production-grade full-stack MERN application (Web Development domain)
Program	Zidio Development
Author	Bharath Kumar C | GitHub: BharathJerry
Date	From: 04.10.2026 To: 04.11.2025
Live demo	https://intell-meet-ai-powered-enterprise-m-wine.vercel.app
API health check	https://intellmeet-ai-powered-enterprise-meeting-adtr.onrender.com/health
Repository	https://github.com/BharathJerry/IntellMeet-AI-Powered-Enterprise-Meeting-Collaboration-Platform
Demo video	https://drive.google.com/file/d/1QraXa-R3jVh0-Gnuq7jaAnS5gfh3diGV/view?usp=drive_link

1. Project Overview

1.1 Vision and objectives

Meetings in many organisations end without a clear record of what was decided or who must act. IntellMeet turns every meeting into a trackable event: people meet over real-time video and chat, the platform produces a summary and action items with owners, and those items become tasks on a team board that everyone can follow.

- Provide reliable real-time video, chat and screen presenting in the browser, with no installation.
- Generate a transcript, a short summary and action items with assignees for each meeting.
- Connect meetings to work: convert action items into tasks and track them on Kanban boards.
- Give teams visibility through a history, an analytics dashboard and exportable reports.

1.2 Target users and use cases

- **Remote and hybrid teams** running stand-ups, reviews and planning sessions.
- **Project leads** who need action items captured and assigned without manual note-taking.
- **Managers** who want a view of meeting frequency and task completion.
- **Guests** who join a single meeting through an invite link.

1.3 Business value

The original brief targets a 40 to 60 percent reduction in meeting follow-up time and a 25 to 40 percent gain in team productivity. These are goals set by the brief; this project provides the mechanism (automatic summaries, action items and task tracking) but does not claim measured percentages.

1.4 Non-functional goals

Goal	Target in the brief	How it is addressed / result
Latency	Under 200 ms for real-time features	Socket.io over WebSocket; measured chat fan-out p95: **818 ms** (see section 6.2)
Concurrency	500 to 5,000 participants	Tested **100** simultaneous socket connections; video is peer-to-peer, so large meetings need a media server (section 6.2)
Availability	99.95 percent	Health endpoint, container health check and uptime monitor. Free-tier hosting cannot guarantee this SLA
Security	JWT, role-based access, encryption	Section 6.1. WebRTC media is encrypted in transit; application-level end-to-end encryption is not implemented
Observability	Logging and monitoring	Structured error handling and health checks. Prometheus/Grafana/Sentry are listed as future work

2. Key Features

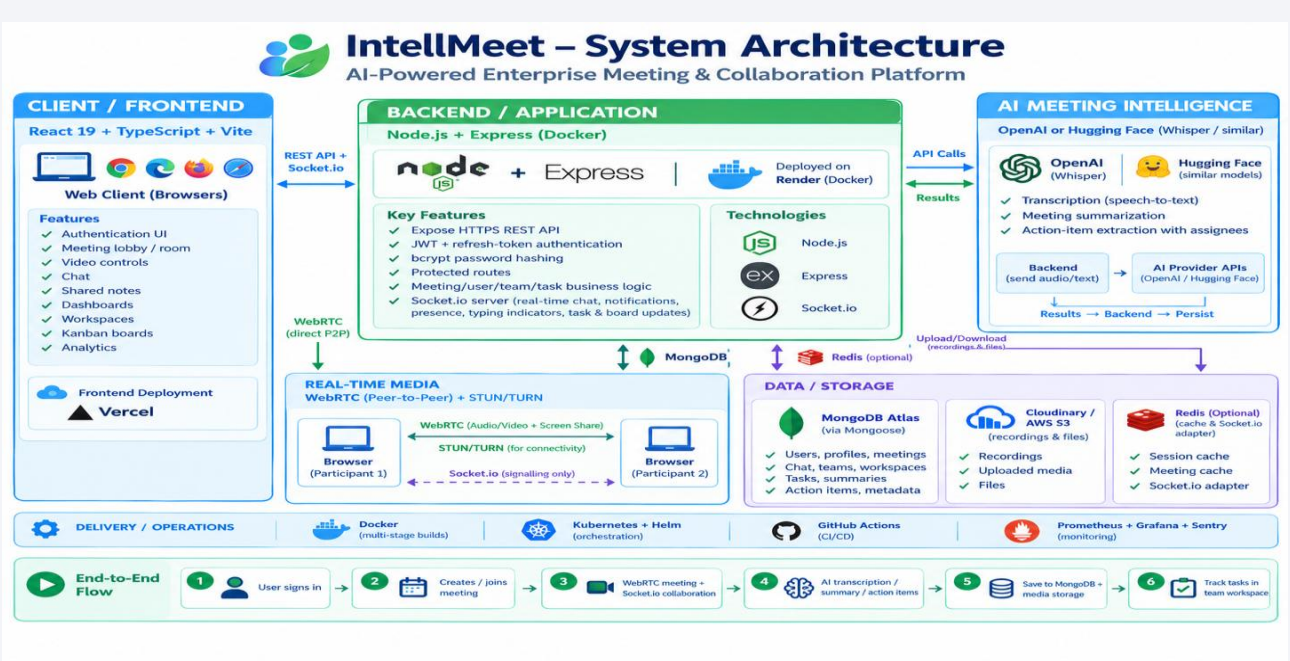
ID	Feature	Description	Acceptance criteria / status
F-01	Authentication and profiles	Signup and login, JWT with refresh-token rotation, optional Google sign-in, profile photo and name, roles (admin, member)	Passwords hashed, tokens expire, role checks tested. Done
F-02	Real-time video meetings	WebRTC video and audio, pre-join lobby, mute and camera controls, Meet-style screen presenting, recording saved to the meeting	Works for small groups (mesh). Recording and presenting tested manually. Done
F-03	AI meeting intelligence	Browser speech transcription, summary and action items with assignees. Works with any OpenAI-compatible provider or a built-in mock mode	Strict JSON validation with one retry; mock mode needs no key. Done
F-04	Real-time chat and collaboration	In-meeting chat with typing indicators, presence, live notifications for tasks, summaries and invitations	Messages reach all participants; covered by socket tests. Done
F-05	Post-meeting dashboard	Searchable history, summary, transcript, recordings, .txt export, one-click task creation	Search, filter and pagination. Done
F-06	Team and project management	Workspaces with roles, invitations by email, Kanban boards with drag-and-drop and live updates	Permissions tested; cards update live for other members. Done
F-07	Analytics and insights	Meetings per day, task status, completion rate, AI usage, CSV export	Charts rendered in the dashboard. Done
F-08	Sharing and scheduling	Invite links, invite a whole team, scheduled meetings, rescheduling, 10-minute reminders, add to calendar	Invite and reminder logic tested. Done

3. Technology Stack

Category	Technology	Rationale / alternatives
Frontend	React 19, Vite, plain CSS	Fast development and builds; small bundle. Alternative: TypeScript with Tailwind
Backend	Node.js, Express	Lightweight, one language across the stack, large ecosystem
Database	MongoDB with Mongoose; GridFS	Flexible documents for meetings and tasks; GridFS stores recordings without another service
Real-time	Socket.io, WebRTC	Socket.io for signalling, chat and presence; WebRTC for peer-to-peer media
AI	OpenAI-compatible API or mock mode	Provider is swappable by configuration; mock mode lets anyone run the demo without a key
Validation / security	Zod, bcrypt, JWT, Helmet, express-rate-limit	Schema validation, hashing, stateless auth, secure headers, abuse protection
Cache / scale (optional)	Redis, Socket.io Redis adapter	Cache-aside for meetings; shared rooms across several servers
Testing	Jest, Supertest, socket.io-client	Unit, integration and real-time tests
DevOps	Docker (multi-stage), GitHub Actions	Reproducible builds; tests and image builds on every push
Hosting	Render (API), Vercel (frontend), MongoDB Atlas	Free tiers with HTTPS, which browsers require for camera access

Differences from the original plan. The brief lists TypeScript, Tailwind, shadcn/ui, TanStack Query, Zustand, Kubernetes with Helm, and Prometheus, Grafana and Sentry. This submission uses plain React and CSS, and replaces Kubernetes with Docker plus managed hosting. The monitoring stack is not implemented; health checks and an uptime monitor are used instead.

4. Architecture



Three communication channels. REST handles data (accounts, meetings, tasks). Socket.io carries real-time events and the WebRTC signalling messages (offers, answers, ICE candidates). Media itself flows directly between browsers over WebRTC, so the server never carries video.

Typical flow. A user creates a meeting through REST, opens the lobby, and joins the Socket.io room. Each newcomer sends WebRTC offers to people already in the room. During the call the browser sends transcribed sentences to the API. When the host requests a summary, the API runs the AI job in the background, stores the summary and action items, and notifies members through a socket event. Action items can then be turned into tasks on a board.

Data model

Collection	Purpose and key fields
User	name, email, hashed password, role, avatar, hashed refresh token
Meeting	title, host, participants, status, scheduledAt, transcript, summary, action items, invite code
Message	meeting, sender, text (chat history)
Task	title, status, assignee, board, workspace, position, due date
Workspace / Board	members with roles; boards belonging to a workspace
recordings (GridFS)	video files with the meeting and uploader in metadata

5. Execution Timeline

Phase	Major deliverables	Milestone / checkpoint
Week 1	Project setup, MongoDB connection, authentication (JWT, refresh tokens, bcrypt), meetings and tasks API, Socket.io rooms, chat and presence	API tested through the browser console; first commit history
Week 2	AI endpoints (transcript, summary, action items), Jest test suites, Redis cache and Socket.io adapter, workspaces, boards and analytics API	About 90 percent test coverage; real-time tests passing
Week 3	React frontend: login, lobby, meeting room (WebRTC), boards, dashboard, history, team pages; recording, avatar upload, Google sign-in	Two-user meeting working end to end
Week 4	Docker, CI pipeline, deployment to Render and Vercel, invite links and scheduling, load and security testing, README and report	Live URL, green CI run, documentation complete

6. Technical Highlights

6.1 Security measures (OWASP Top 10)

OWASP risk	Mitigation in this project
A01 Broken access control	Every resource checks membership and role; non-members receive 403. Covered by integration tests for meetings, boards, recordings and invites
A02 Cryptographic failures	bcrypt (cost 12) for passwords; secrets only in environment variables; refresh tokens stored as SHA-256 hashes; HTTPS on all hosted parts
A03 Injection	Zod validates every request body; Mongoose queries are parameterised; text search uses a MongoDB text index, not user-built regular expressions
A05 Security misconfiguration	Helmet headers, CORS allow-list taken from configuration, body size limits, production error messages hide internals
A07 Authentication failures	15-minute access tokens, rotating refresh tokens, rate limiting on auth routes, Google ID tokens verified on the server
A08 Integrity failures	Lock files committed; CI installs with npm ci and runs tests on every push
Uploads	Avatars must be real JPEG files (signature check) up to 200 KB; recordings limited to 50 MB and members only

Not implemented: application-level end-to-end encryption, centralised audit logging and automated vulnerability monitoring.

6.2 Performance and scalability

- **Caching:** meeting lookups use a Redis cache-aside layer (60 second expiry, cleared on every change). Responses carry an X-Cache header showing HIT or MISS; permission checks still run on a hit.
- **Scaling real-time:** presence is stored on each socket and read with fetchSockets, so the Socket.io Redis adapter lets several server instances share rooms.
- **Database:** pagination, a text index for search, and background processing of AI jobs so requests return immediately.
- **Frontend:** small bundle (no UI framework), pages load only the data they need, optimistic updates when moving board cards.

Test	Load	Throughput / p95	Success rate	Notes
HTTP API mix	10 users, 30 s	**12.7 req/s & p95 up to 2,294 ms**	**100% (386 requests)**	Slowest: analytics overview (p50 785 ms)
HTTP API mix	25 users, 30 s	**22.1 req/s & p95 up to 2,769 ms**	**100% (678 requests)**	Latency rises with load
HTTP API mix	50 users, 30 s	**28.2 req/s & p95 up to 3,099 ms**	**100% (900 requests)**	Even /health p95 reached 2,288 ms
Socket connections	20	connect **p95 2,528 ms / join p95 1,157 ms**	**100% (20 of 20)**	
Socket connections	50	connect **p95 1,965 ms / join p95 2,404 ms**	**100% (50 of 50)**	
Socket connections	100	connect **p95 2,747 ms / join p95 4,737 ms**	**100% (100 of 100)**	Join time grows with the number joining at once
Chat fan-out to all	10 messages × 20 / 50 / 100	**p95 717 / 717 / 818 ms**	**100% (200/200, 500/500, 1,000/1,000)**	Every message reached every connection
Lighthouse (mobile)	Vercel URL	Perf **95** / A11y **94** / Best Practices **100** / SEO **100**		
Lighthouse (desktop)	Vercel URL	Perf **96** / A11y **94** / Best Practices **100** / SEO **100**		

6.3 Testing

The backend has unit, integration and real-time tests written with Jest, Supertest and socket.io-client. Integration suites each use a separate test database and refuse to run unless its name contains "test". The CI pipeline runs the full suite against a MongoDB container. At the last full run there were **104 tests in 10 suites** with about **94.13 percent statement coverage**.

Area	What is verified
Unit	AI mock parser, AI provider handling with a fake provider (valid JSON, retry on bad JSON, provider errors, chunking), tokens, validation and error middleware, cache helpers
Integration	Authentication and refresh rotation, meetings and permissions, tasks, workspaces, boards and analytics, recordings, avatar validation, invites and reminders
Real-time	Socket authentication, joining rules, chat, typing, presence, media state, WebRTC signalling relay, notifications
Manual	Video between devices on different networks, screen presenting, recording, Google sign-in, deployment smoke test

6.4 Challenges faced and how they were solved

Problem	Cause	Solution
Refresh token rotation did not invalidate old tokens	Tokens issued in the same second were identical found by a test	Added a unique token ID tokens are hashed with SHA-256 because bcrypt only reads the first 72 bytes
Parallel tests wiped each other's data	Several test files shared one database	Each integration suite uses its own database name, with a guard that requires "test" in the name
Screen sharing did not feel like Google Meet	First version only swapped the camera track	Redesigned so the screen travels over a second connection per viewer; it takes the main stage while cameras stay in a strip
Presence broke with more than one server	Participant list lived in server memory	Stored on socket data and read with fetch Sockets so the Redis adapter works
Could not reach MongoDB Atlas from the development laptop	Network-level TLS and allow-list issues	Isolated the problem with a connection test script; used local MongoDB for development and Atlas for production
CI failed at npm ci	Lock file written by Node 24 CI used Node 20	Aligned Node versions in the workflow and Docker files
Browser blocked requests (CORS)	Allowed origin had a trailing slash, and Vercel creates several URLs	Origins are read from a comma-separated list with trailing slashes removed
Frontend called localhost in production	Vite environment variables are fixed at build time	Set VITE_API_URL on the host and rebuilt; added a production fallback
Login blocked with HTTP 429 on mobile	Rate limiter saw the hosting proxy's address for everyone	Enabled trust proxy and counted only failed attempts
Slow first request on free hosting	Render free plan sleeps when idle	Uptime monitor pings the health endpoint

7. Deployment and Operations

Item	Details
Platforms	Frontend on Vercel API on Render (Docker image built from backend/Dockerfile); database on MongoDB Atlas
Containers	Multi-stage Docker files (production dependencies only, non-root user, health check). docker-compose.yml starts MongoDB, Redis, API and frontend locally
CI/CD	GitHub Actions on every push and pull request: backend tests against a MongoDB service, frontend build, Docker image builds. Render and Vercel redeploy automatically from the main branch
Configuration	All secrets and URLs are environment variables; .env files are git-ignored; render.yaml documents the service and generates the JWT secrets
Monitoring	/health endpoint, container health check, uptime monitor, platform logs
Demo access	A one-click demo account on the login page; mock AI mode so no private keys are needed

8. Visuals

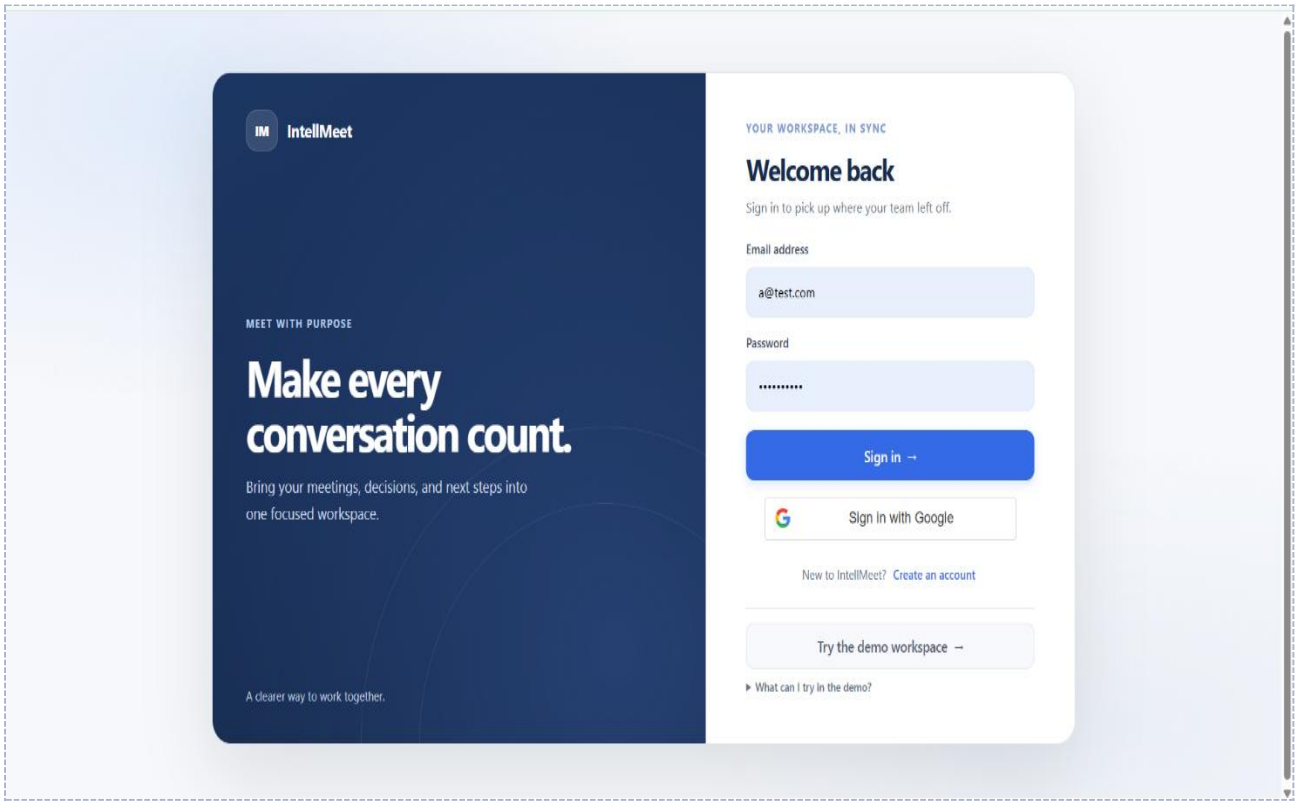


Figure 1: Login page with the demo account button

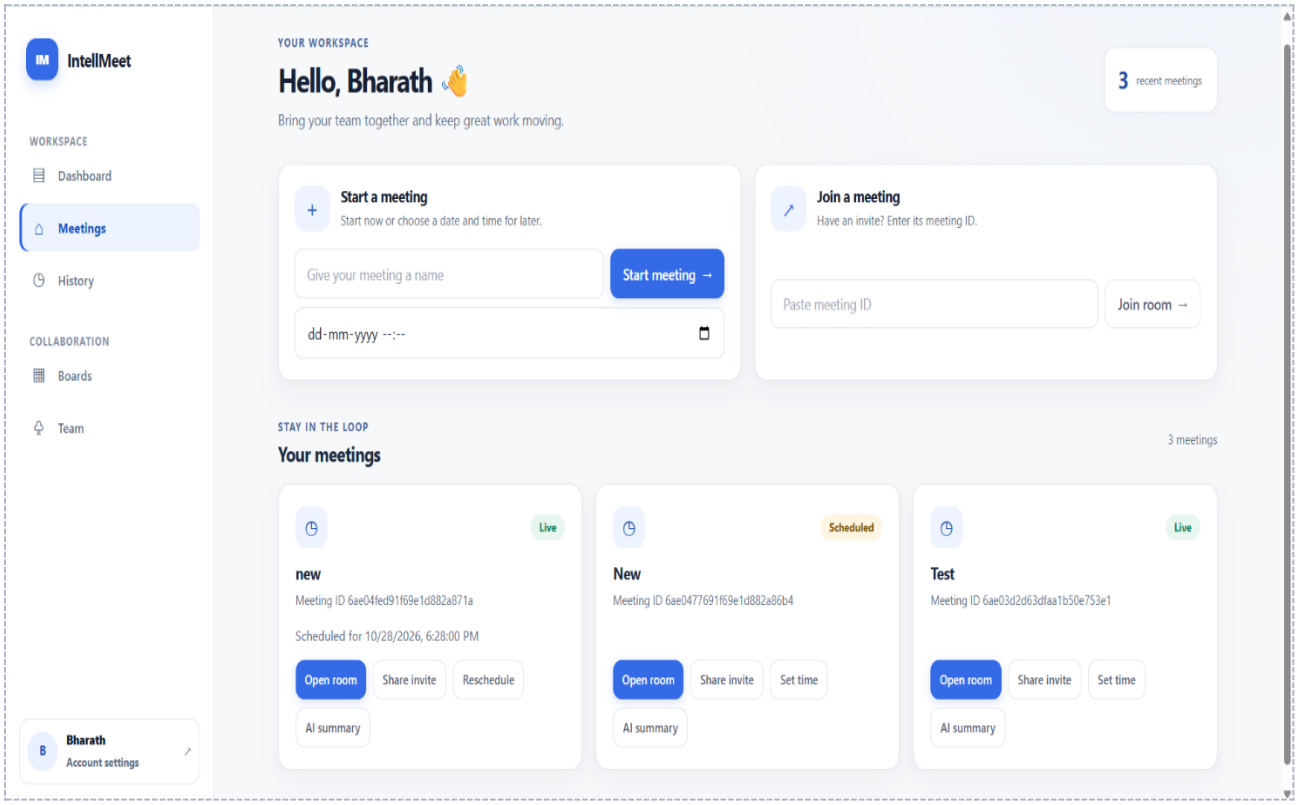


Figure 2: Meetings page with upcoming scheduled meetings and the Share panel

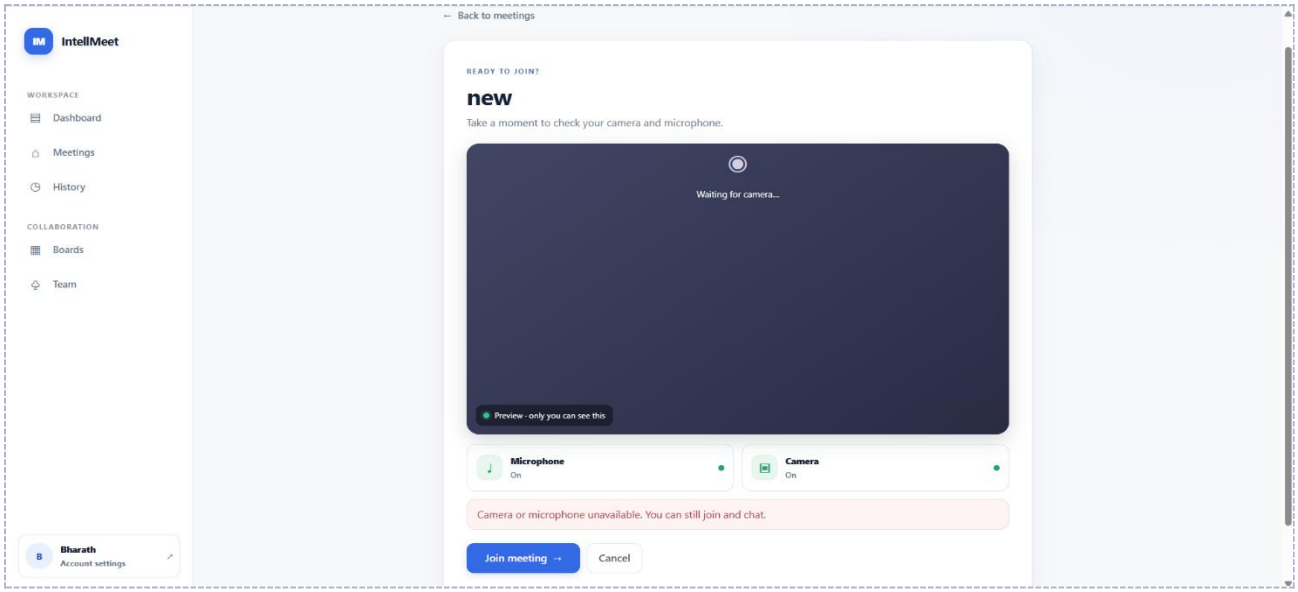


Figure 3: Pre-join lobby with camera preview

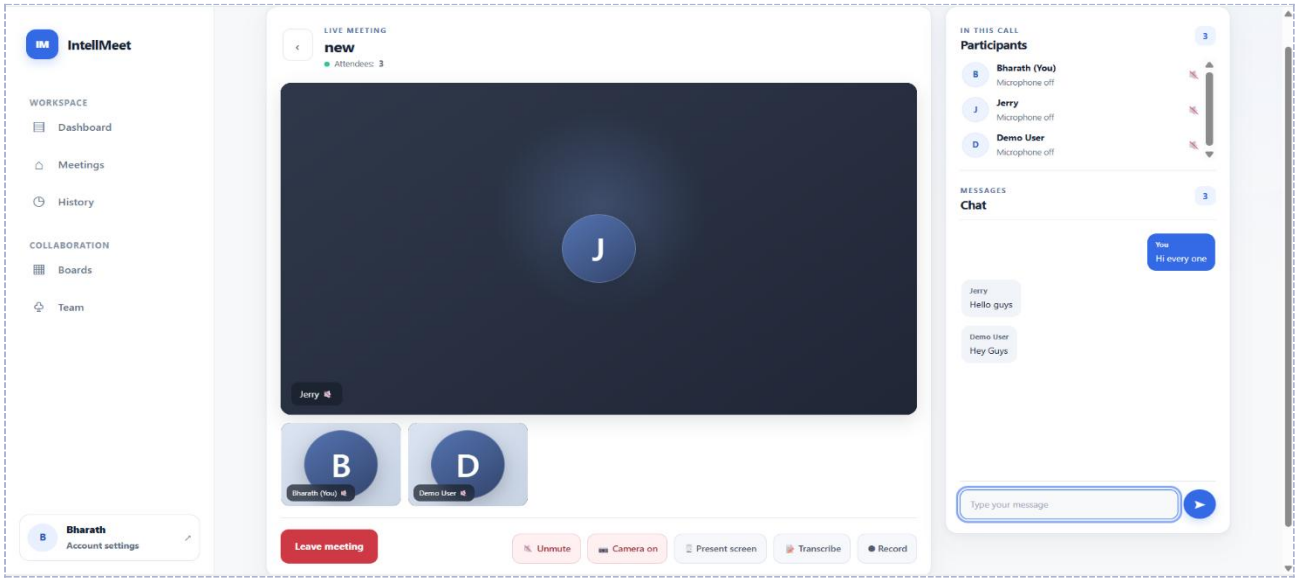


Figure 4: Meeting room with several participants and chat

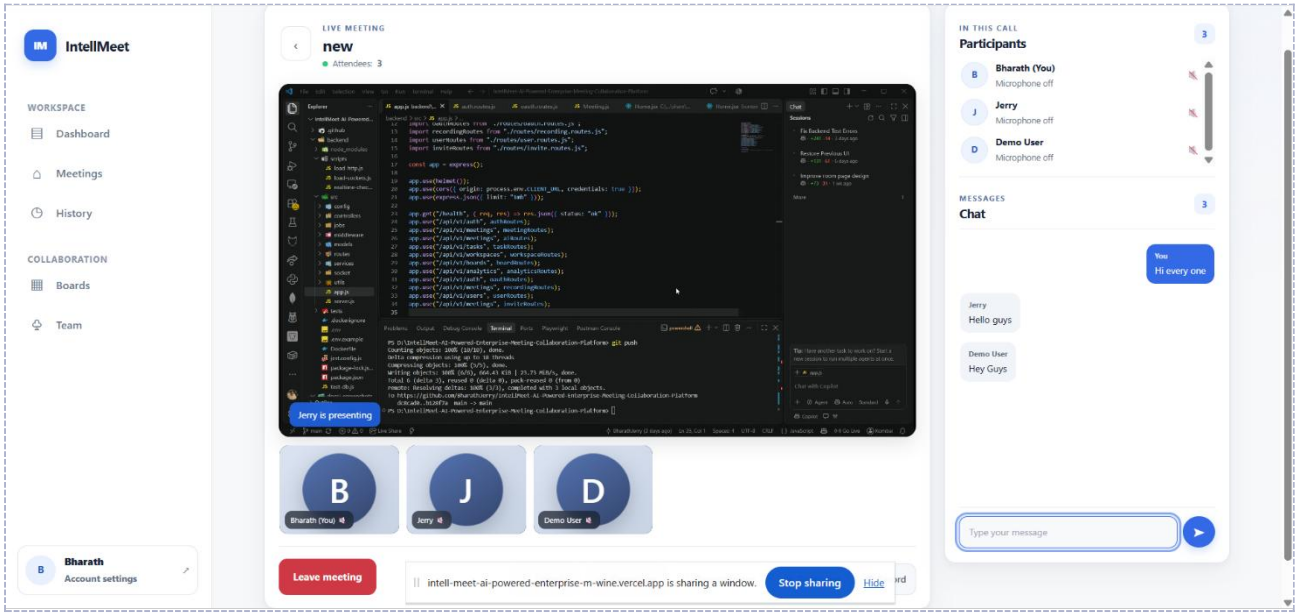


Figure 5: A participant presenting their screen

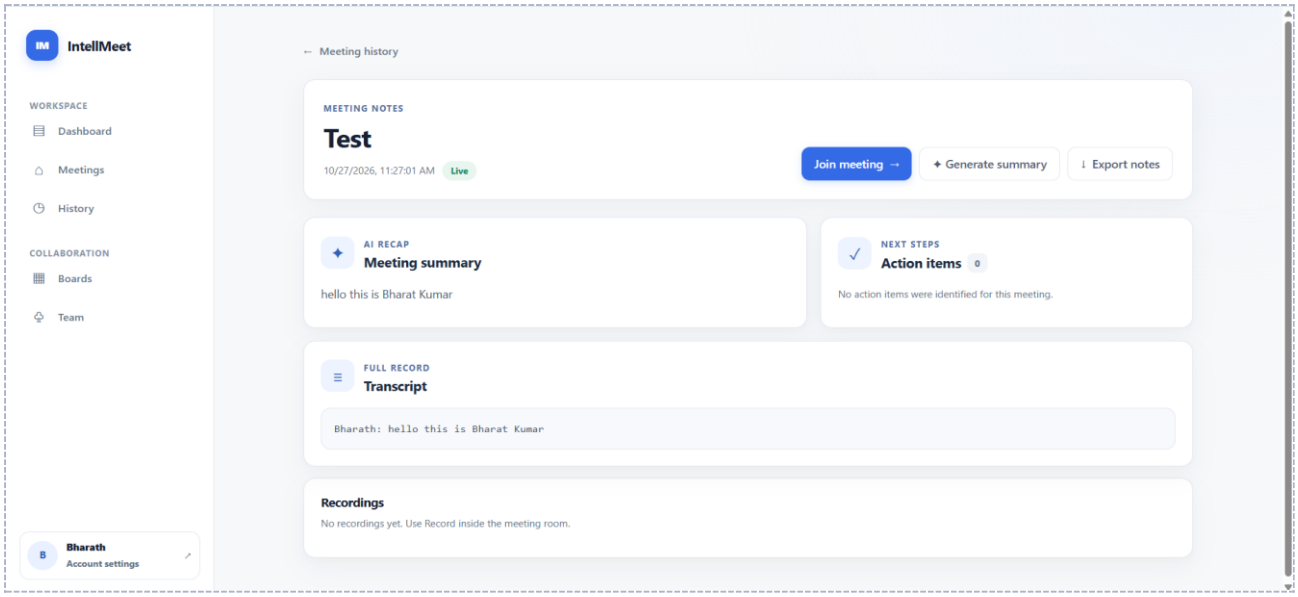


Figure 6: Meeting details with AI summary, action items and transcript

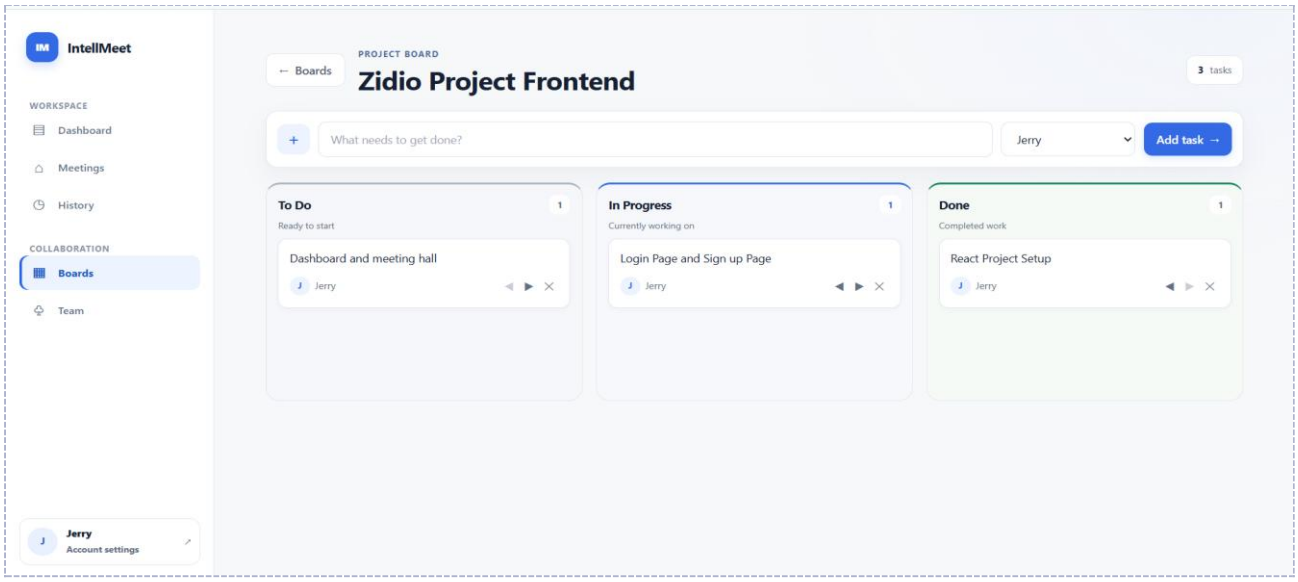


Figure 7: Kanban board with cards in three columns

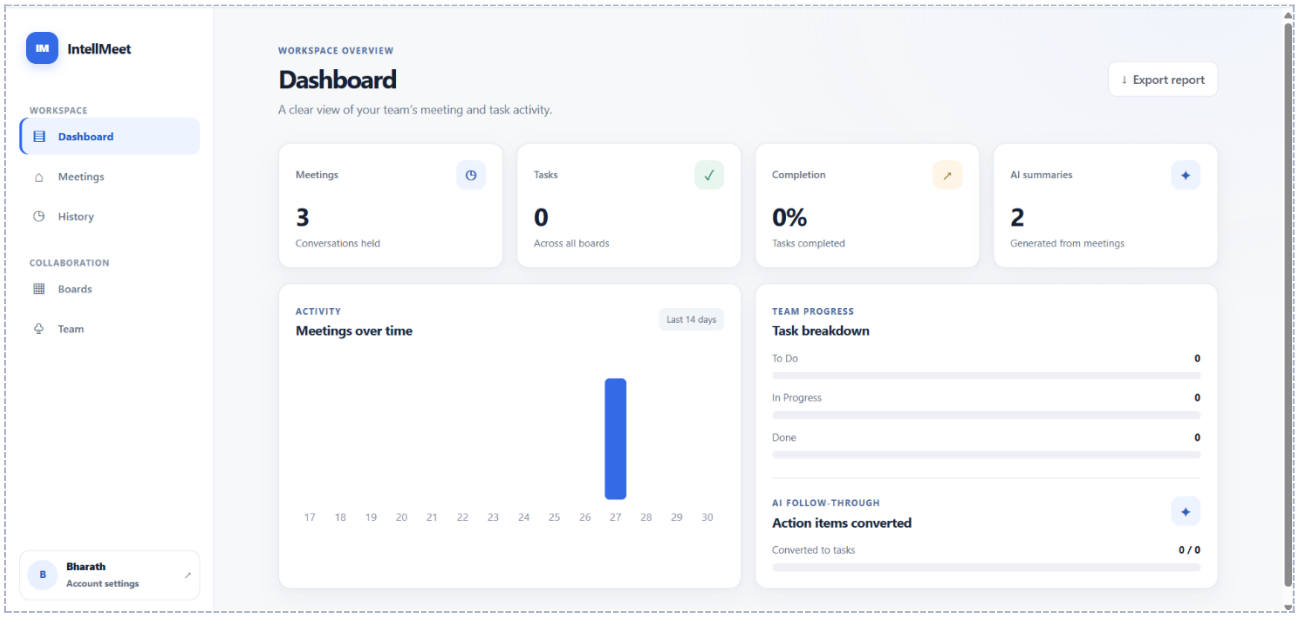


Figure 8: Analytics dashboard

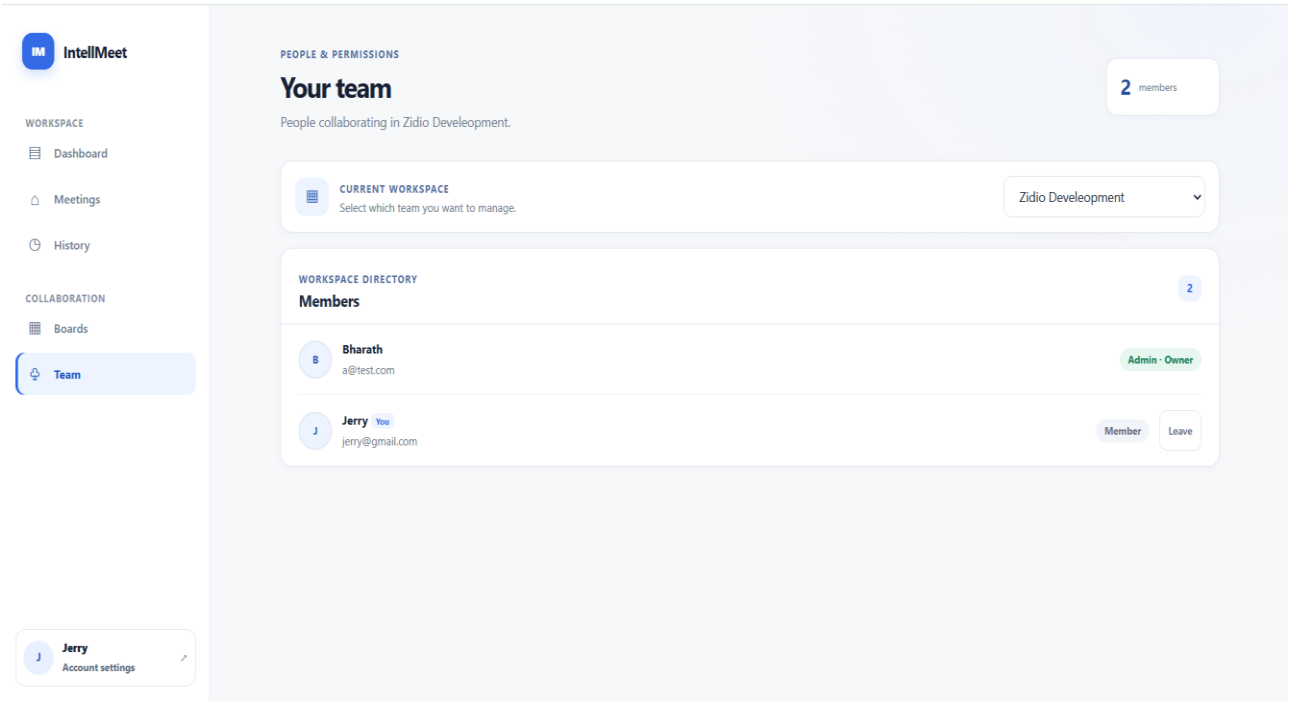
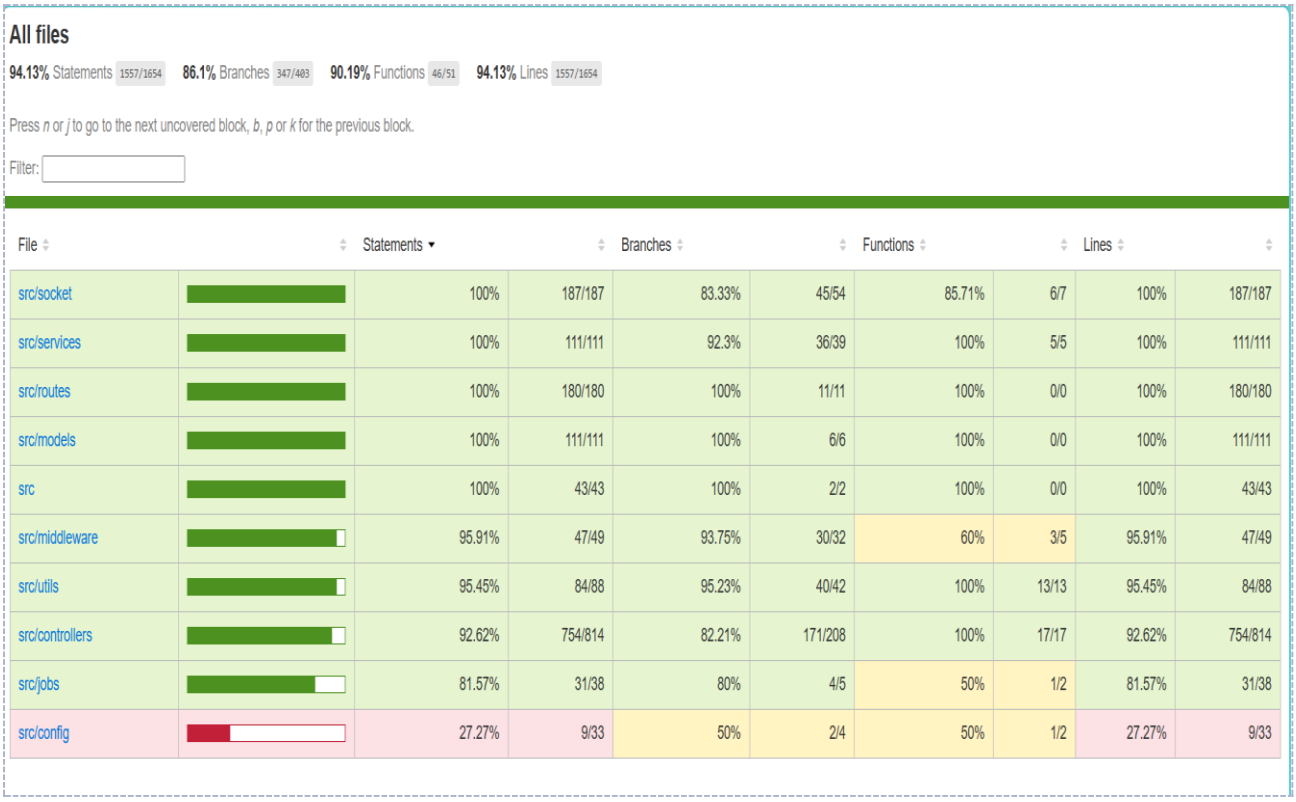


Figure 9: Team



All files

94.13% Statements 1557/1654 **86.1%** Branches 347/403 **90.19%** Functions 46/51 **94.13%** Lines 1557/1654

Press *n* or *j* to go to the next uncovered block, *b*, *p* or *k* for the previous block.

Filter:

File	Statements		Branches		Functions		Lines	
src/socket	100%	187/187	83.33%	45/54	85.71%	6/7	100%	187/187
src/services	100%	111/111	92.3%	36/39	100%	5/5	100%	111/111
src/routes	100%	180/180	100%	11/11	100%	0/0	100%	180/180
src/models	100%	111/111	100%	6/6	100%	0/0	100%	111/111
src	100%	43/43	100%	2/2	100%	0/0	100%	43/43
src/middleware	95.91%	47/49	93.75%	30/32	60%	3/5	95.91%	47/49
src/utils	95.45%	84/88	95.23%	40/42	100%	13/13	95.45%	84/88
src/controllers	92.62%	754/814	82.21%	171/208	100%	17/17	92.62%	754/814
src/jobs	81.57%	31/38	80%	4/5	50%	1/2	81.57%	31/38
src/config	27.27%	9/33	50%	2/4	50%	1/2	27.27%	9/33

Figure 10: Test coverage table

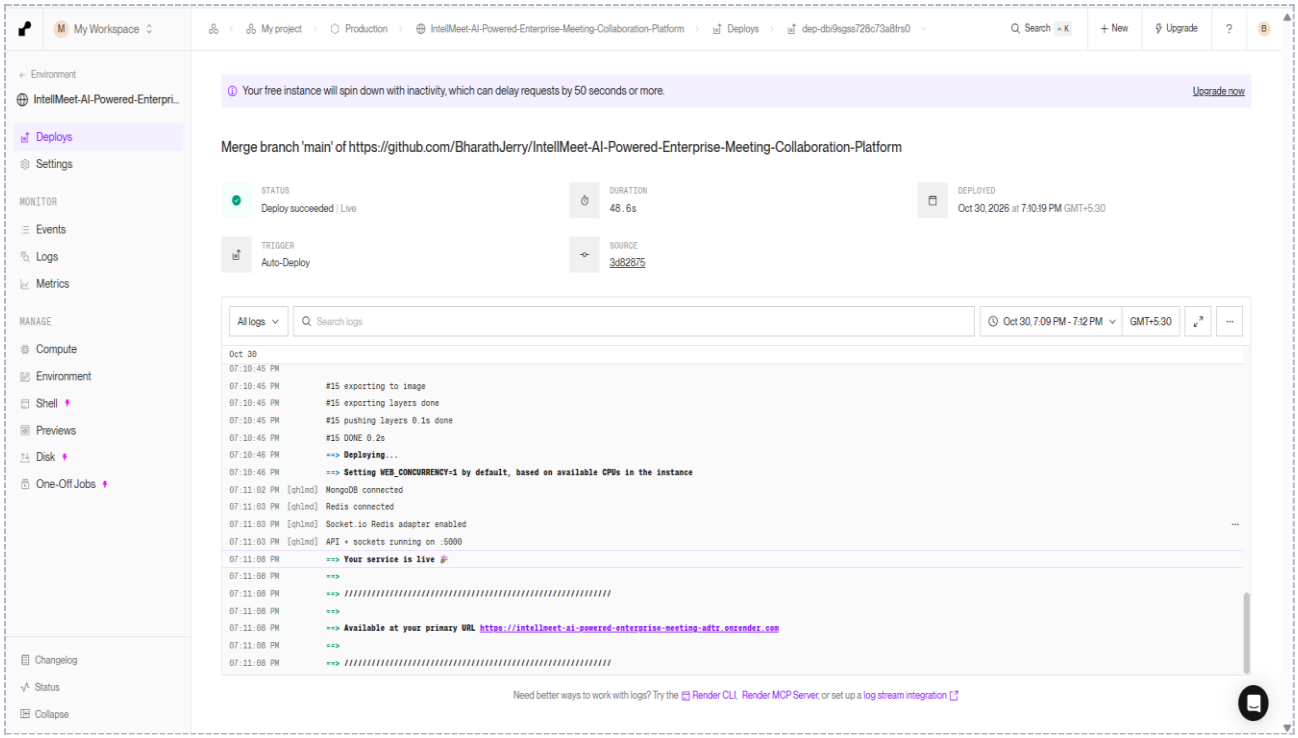


Figure 11: Render dashboards showing the live deployment

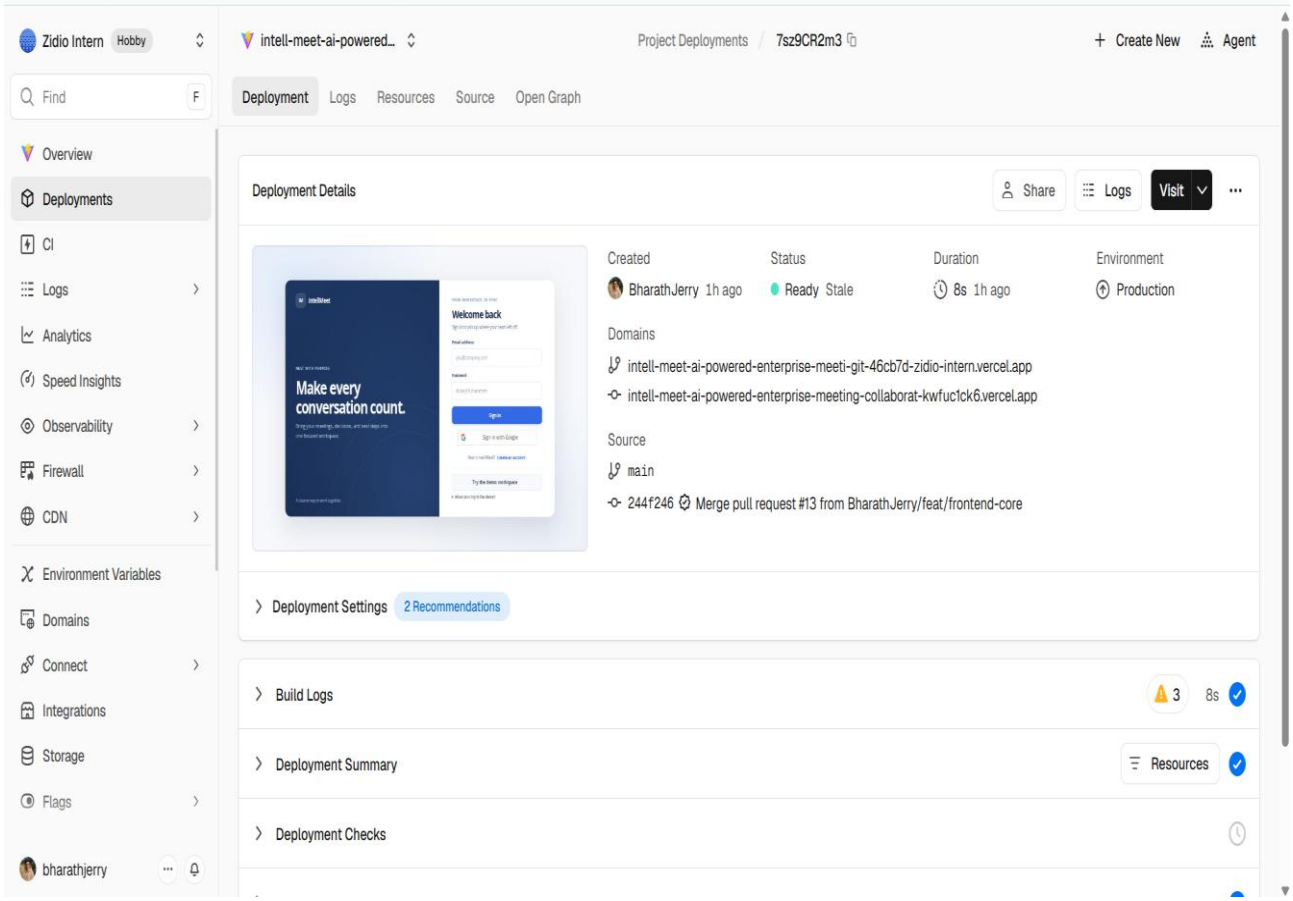


Figure 12: Vercel dashboards showing the live deployment

9. Personal Reflection

Key learnings

Building IntellMeet taught me how much of a real-time application happens outside the code editor. WebRTC video only works because a separate signaling channel passes offers and ICE candidates between browsers and understanding that split between Socket.io, REST and peer-to-peer media was the lesson. Writing tests early paid off: a failing test exposed a refresh-token bug where tokens issued in the second were identical which I would have missed by testing. Deployment was harder than development. Problems that never appeared on my laptop, such as a CORS error caused by a trailing slash, a Vite variable that is fixed at build time a CI failure from Node versions and HTTP 429 errors caused by a hosting proxy each taught me to read logs of guessing. I also learned to be honest about limits: video, between browsers suits meetings and the hosting tier slows down under load so I measured both and documented limits, than hiding them. If I did this again I would set up deployment and CI in the week not the last.

Industry best practices applied

- Layered backend structure (routes, controllers, services, models) with centralised error handling and input validation.
- Automated tests and a CI pipeline; separate test databases and guards against touching real data.
- Secrets in environment variables, least-privilege access checks, rate limiting and secure headers.
- Optional components (Redis, Google sign-in, real AI provider) that degrade gracefully when not configured.
- Conventional commits, feature branches and pull requests.

Future roadmap

- Replace peer-to-peer mesh with a media server (LiveKit or mediasoup) for meetings of 50 or more people, and add a managed TURN service.
- Server-side recording and cloud storage (S3 or Cloudinary) with retention rules.
- Email and push notifications for reminders; calendar synchronisation.
- Application-level end-to-end encryption for chat and transcripts.
- Kubernetes with Helm for autoscaling, plus Prometheus, Grafana and Sentry for observability.
- Migrate the frontend to TypeScript and add end-to-end browser tests.